

PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Daniel Felipe Sarmiento Pilonieta-2202797

Brandon Ferney Caballero Barajas - 2202809

https://github.com/Danielsarmiento24/CommII_B1_G4/tree/Practica_1/

Abstract

This laboratory report describes the design, implementation, and validation of custom processing blocks in GNU Radio using Python, as part of the study of Software Defined Radio (SDR) systems. Two blocks were developed: an Accumulator, used to integrate signal values over time, and a Differentiator, applied to highlight changes between consecutive samples. These tools allowed the extraction of deterministic parameters such as amplitude, frequency, phase, and period, as well as probabilistic measures including mean, variance, and RMS. Their application made it possible to analyze signals in real time and to evaluate the effect of noise and its mitigation through statistical methods. The results confirm the correct operation of the implemented blocks and provide with practical experience in combining theoretical concepts of communications with hands-on programming in GNU Radio.

• Introducción

En el estudio de las comunicaciones modernas, la radio definida por software (SDR) se ha consolidado como una herramienta fundamental para comprender y experimentar con sistemas de transmisión y recepción de señales. A diferencia de los equipos de hardware rígidos, SDR permite modificar el comportamiento del sistema mediante software, ofreciendo un entorno flexible y dinámico para la exploración de conceptos teóricos y su aplicación práctica.

GNU Radio, en particular, constituye una de las plataformas más utilizadas en la formación de ingenieros en telecomunicaciones, ya que combina bloques predefinidos con la posibilidad de crear módulos personalizados en Python. Esta característica facilita al estudiante no solo el uso de herramientas ya establecidas, sino también el desarrollo de bloques propios que extienden las capacidades del software, fomentando así el aprendizaje activo y la investigación aplicada.

En este laboratorio se propone la implementación de bloques programados en Python, específicamente un acumulador y un diferenciador, que permiten extraer información relevante de las señales procesadas. A través de estos bloques es posible calcular parámetros determinísticos como la amplitud, frecuencia, fase y período, así como parámetros probabilísticos relacionados con el análisis estadístico de señales, tales como media, varianza y valor RMS.

El propósito central de la práctica es integrar los conceptos aprendidos en clase teórica con la experimentación práctica, logrando una comprensión más profunda del comportamiento de señales determinísticas y aleatorias,

además de fortalecer competencias en el uso de herramientas como GitHub. Con ello, se busca afianzar las bases necesarias para el análisis, diseño y optimización de sistemas de comunicación más avanzados.

• Procedimiento

Para iniciar la práctica se configuró el entorno de trabajo a través de la creación de un repositorio en GitHub, en el cual se estableció una rama principal denominada Practica1. Dentro de esta rama se organizaron dos directorios.

Posteriormente, se procedió con la creación de bloques personalizados en GNU Radio utilizando el entorno GNU Radio Companion (GRC). Para este fin, se empleó el bloque denominado Python Block, el cual permite programar funciones específicas no incluidas dentro de los bloques estándar. La programación se realizó en lenguaje Python, siguiendo como apoyo la guía proporcionada en la práctica “*Comunicaciones digitales basadas en radio definida por software*.”¹ y la documentación oficial de GNU Radio².

Parámetros de simulación:

Frecuencia de muestreo: $f_s = 1000 \text{ Hz}$;

Señal cuadrada: $f = 1 \text{ kHz}$, amplitud $A = 1A$,
 $offset = -0.5$

Señal senoidal: $f_0 = 5 \text{ Hz}$, amplitud $A = 1A$

Ruido gaussiano: entre $\sigma = 0.01$ y $\sigma = 1$

El primer bloque implementado correspondió al Acumulador, cuyo propósito fue realizar la suma acumulada de las muestras de la señal de entrada según la expresión $y[n]=y[n-1]+x[n]$. Con el fin de evitar la cancelación entre valores positivos y negativos al trabajar con una señal cuadrada simétrica, se aplicó un desplazamiento (offset) en la entrada. De esta manera, fue posible visualizar claramente la evolución acumulada de la señal, la cual se comparó en tiempo real con la forma de onda original mediante un QT GUI Time Sink.

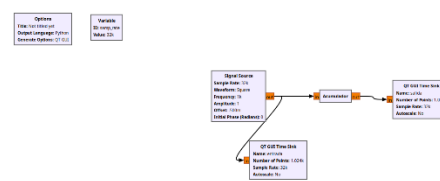


Figura 1. Bloque acumulador GNU Radio.

En segundo lugar, se desarrolló el Diferenciador, un bloque encargado de calcular la diferencia entre muestras consecutivas, lo cual equivale a una operación de derivación discreta descrita por la relación $y[n] = x[n] - x[n-1]$. Para comprobar su funcionamiento se utilizó una onda senoidal como señal de entrada, verificando que la salida representara correctamente la pendiente instantánea de la señal, lo que evidencia el comportamiento esperado en este tipo de operaciones.

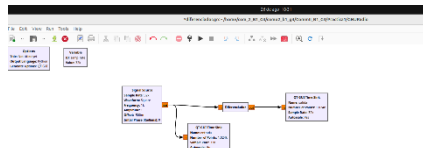


Figura 2. Bloque Diferenciador GNU Radio.

Finalmente, se diseñó un bloque de Estadísticas de Señal, cuyo objetivo fue calcular en tiempo real parámetros probabilísticos que describen el comportamiento de señales determinísticas y aleatorias. Dentro de este bloque se implementaron funciones para calcular la media, la media cuadrática, el valor RMS, la potencia promedio y la desviación estándar.

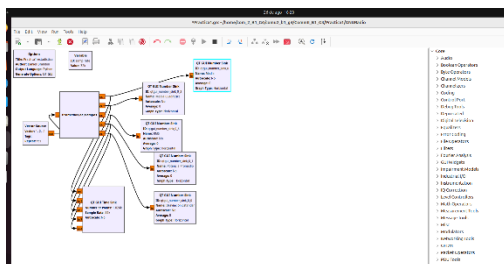


Figura 3. Bloque Promedios de Tiempo GNU Radio.

En la última simulación se implementó ruido gaussiano en el bloque de promedios de tiempo con el fin de reducir el efecto del ruido gaussiano, la señal procesada fue dividida en ventanas de tamaño fijo, sobre las cuales se calcularon las estadísticas, logrando así estimaciones más estables y representativas.

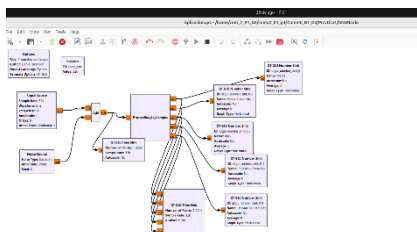


Figura 4. Bloques promedios de Tiempo con ruido gaussiano GNU Radio.

- **Análisis de resultados**

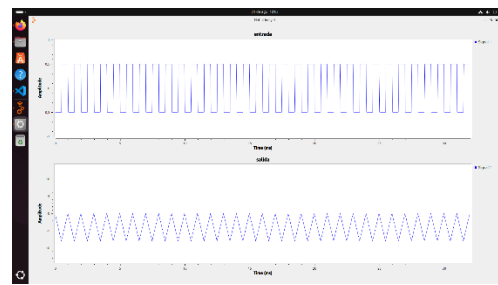


Figura 5. Bloque acumulador de una señal cuadrada

En la parte superior se observa la señal cuadrada de entrada con niveles $+0.5$ y -0.5 y frecuencia fundamental $f_0=1$ kHz. En la parte inferior aparece la salida del acumulador, que corresponde a la suma acumulativa de cada muestra de la señal de entrada. El acumulador se comporta como un integrador discreto: durante cada medio-período suma repetidamente el mismo valor (positivo o negativo) y por eso la salida adopta una forma triangular periódica.

En la figura la salida muestra valores entre 0 y -8, lo cual es coherente con el modelo si la ventana de visualización inicia en la fase donde primero se acumula el tramo negativo (desde $y[0] = 0$ baja hasta -8 y luego vuelve a 0). Dado que los valores medidos (magnitud ≈ 8) coinciden con el valor teórico calculado, se concluye que el bloque acumulador cumple correctamente su función: integra discreta y linealmente la señal de entrada y produce la forma triangular esperada bajo condiciones de muestreo y amplitud dadas.

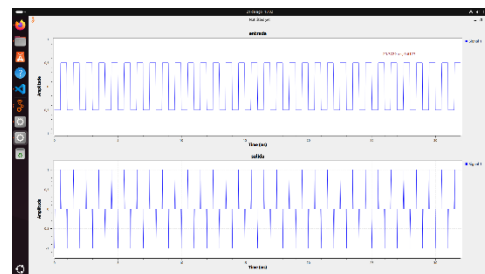


Figura 6. Bloque Diferenciador de una señal cuadrada.

En la parte superior se presenta la señal de entrada, una onda cuadrada con niveles de $+0.5$ y -0.5 y frecuencia fundamental de $f_0=1$ kHz. En la parte inferior se observa la salida del bloque diferenciador, la cual corresponde a la variación discreta entre muestras consecutivas de la señal de entrada.

El comportamiento obtenido es el esperado: durante los tramos constantes de la onda cuadrada la salida permanece cercana a cero, mientras que en cada transición aparecen impulsos positivos o negativos que representan los cambios abruptos de la señal. La amplitud de dichos impulsos coincide aproximadamente con la magnitud del salto de la onda cuadrada ($\Delta x = 1$), lo que confirma que el bloque resalta las discontinuidades de la señal.

Después de analizar de manera individual el comportamiento del bloque acumulador y del bloque diferenciador, se realizó una comparación conceptual con el fin de resaltar las diferencias más relevantes entre ambos. La Tabla 1 sintetiza esta comparación, mostrando de manera clara sus funciones principales, respuesta frente a señales periódicas, sensibilidad al ruido y aplicaciones típicas.

Característica	Acumulador	Diferenciador
Función principal	Realiza la suma acumulativa de las muestras (integrador discreto).	Calcula la diferencia entre muestras consecutivas (derivada discreta)
Efecto sobre señales periódicas	Convierte una señal cuadrada en una forma triangular.	Convierte una señal cuadrada en impulsos en las transiciones.
Sensibilidad al valor medio	Si la señal tiene un valor medio distinto de cero, la salida crece indefinidamente (posible desbordamiento).	Si la señal permanece constante, la salida tiende a cero.
Aplicaciones típicas	Estimación de energía acumulada, filtros integradores, control digital.	Detección de bordes o transiciones, realce de cambios abruptos, procesamiento de imágenes y comunicaciones.
Sensibilidad al ruido	Menos sensible al ruido de alta frecuencia, ya que lo suaviza en la suma.	Muy sensible al ruido, ya que amplifica las variaciones rápidas.

Tabla 1. Comparación conceptual entre el bloque acumulador y diferenciador.

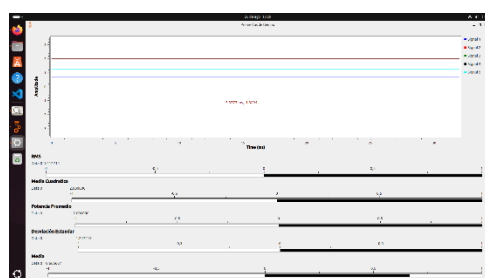


Figura 7. Simulación Promedios de tiempo

En la simulación realizada con el bloque de promedio de tiempo se calcularon diferentes parámetros estadísticos y determinísticos de la señal de entrada. En la parte superior se observan las salidas de cinco señales de prueba, cuyos valores promedio permanecen constantes en el tiempo, como corresponde al análisis de señales periódicas estables.

Los resultados obtenidos fueron los siguientes:

RMS: 1.414, valor coherente con el esperado para una señal de amplitud ± 2 , dado que $RMS = \frac{A}{\sqrt{2}}$.

Media cuadrática: 2.0, correspondiente al valor cuadrático medio de la señal.

Potencia promedio: 2.0, coincidiendo con la media cuadrática y validando que la potencia equivale al cuadrado del RMS.

Desviación estándar: 1.247, lo que refleja la dispersión de la señal respecto a su media.

Media: 0.667, indicando la presencia de una componente DC positiva en la señal.

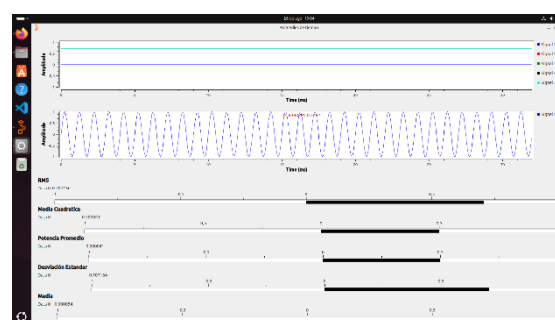


Figura 8. Simulación con ruido gaussiano 0.01

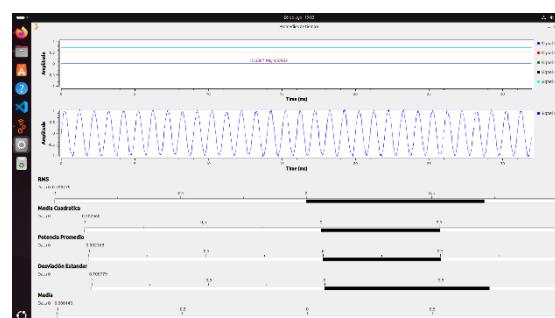


Figura 9. Simulación con ruido gaussiano 0.05

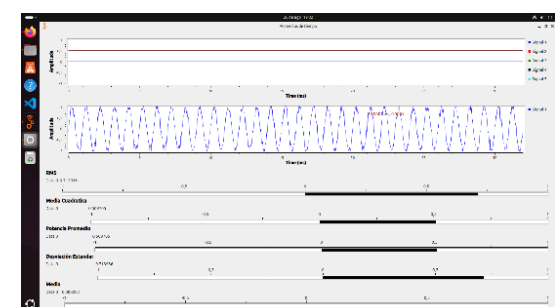


Figura 10. Simulación con ruido gaussiano 0.1

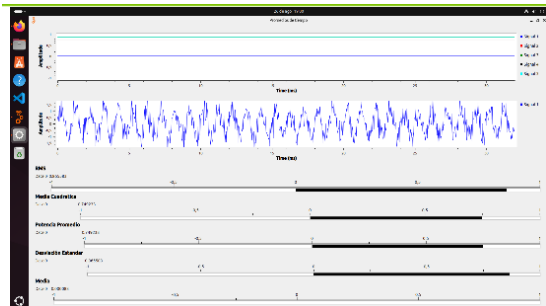


Figura 12. Simulación con ruido gaussiano 0.5

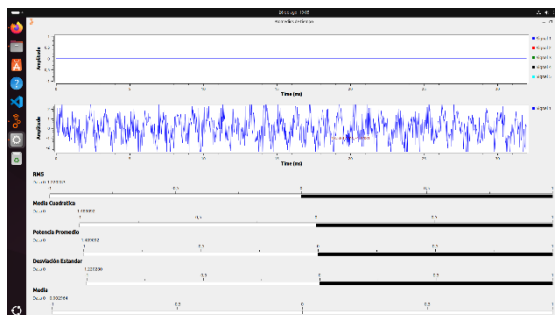


Figura 13. Simulación con ruido gaussiano 1

Al introducir ruido gaussiano moderado, la forma de onda comienza a presentar pequeñas variaciones alrededor de la senoidal original. Como consecuencia, los valores estadísticos se mantienen próximos a los teóricos, pero con ligeras variaciones: la desviación estándar aumenta y la media presenta una oscilación mínima. Esto indica que el ruido empieza a dispersar las muestras, aunque el bloque de promedios aún conserva la estabilidad de los valores centrales.

En la simulación con ruido gaussiano de mayor intensidad, la dispersión es mucho más notoria: la señal se ve distorsionada y los parámetros estadísticos reflejan ese comportamiento.

Cuando se incrementa la intensidad del ruido a un valor de 1, la distorsión en la senoidal se hace mucho más notoria. La potencia promedio asciende hasta ≈ 1.48 , reflejando un aumento significativo de la energía de la señal debido al ruido. La desviación estándar también proporción, lo que evidencia una mayor dispersión de las muestras alrededor de la media. A pesar de ello, la media sigue siendo cercana a cero, lo cual concuerda con la simetría del ruido gaussiano.

• Conclusión

La implementación de bloques personalizados en GNU Radio mediante Python permitió comprobar la aplicabilidad de los conceptos teóricos vistos en clase, especialmente en lo relacionado con parámetros determinísticos y probabilísticos de las señales. Esto fortaleció la integración entre teoría y práctica, brindando una experiencia directa en el diseño, programación y

análisis de operadores básicos en procesamiento digital de señales.

El análisis del bloque acumulador evidenció un crecimiento acumulativo en la salida y la posibilidad de desbordamiento numérico al trabajar con señales de valor medio distinto de cero. Esto permitió contrastar el comportamiento ideal de la simulación con las limitaciones que se presentan en implementaciones digitales reales. Por su parte, el bloque diferenciador mostró su capacidad para detectar cambios abruptos en señales cuadradas, generando pulsos en los flancos y manteniéndose cercano a cero en los tramos de nivel constante, validando su utilidad como detector de bordes y resaltando su sensibilidad al ruido.

Finalmente, el bloque de estadísticas demostró la relevancia del promediado por ventanas para estabilizar las estimaciones de parámetros como la media, la potencia promedio y la desviación estándar. Los resultados confirmaron que, al introducir ruido gaussiano, la potencia promedio y la dispersión de la señal aumentan de forma proporcional a la intensidad del ruido, lo cual constituye un hallazgo práctico para el diseño de sistemas de comunicaciones más robustos.

En conjunto, la práctica no solo permitió validar los fundamentos teóricos, sino también reconocer limitaciones prácticas como la saturación, la amplificación del ruido y la dependencia de parámetros de configuración. Estos aspectos son críticos para la transición de simulaciones en software hacia implementaciones reales en hardware SDR.

REFERENCIAS

- [1] H. Ortega Boda, *Comunicaciones digitales basadas en radio definida por software*. (pp. 16-22).
- [2] GNU Radio Project, “GNU Radio Documentation,” *GNU Radio Foundation*, 2025. [En línea]. Disponible en: <https://www.gnuradio.org/docs/>